

simulation [from ROBOTIS](#), which has a small simulated world with 3 Turtlebot3 mobile robots, each running its own ROS 1 navigation stack.

After successful builds for both ROS 1 and ROS 2 workspaces, the simulation can be launched following [these instructions](#), which also includes a ROS 2 `free_fleet` server, publishing fleet state messages and accepting mode and navigation requests over ROS 2 messages and topics.

ROS 2 Navigation Stack

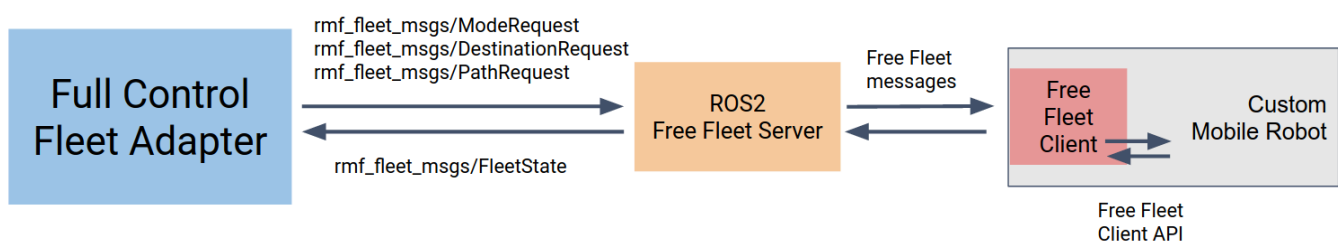
An implementation for a robot using ROS 2 would be similar to a ROS 1 navigation stack described earlier. At this time, the ROS 2 `free_fleet` client is still under development. This section will be updated once the refactoring, implementation and testing has been completed.

The same ready `free_fleet` server implementation in the repository will work in this scenario, as the interfaces provided by the fleet adapters are still the same ROS 2 messages and topics.

If required in the meantime, users can implement their own `free_fleet` client, by working with the `free_fleet` library that contains the base implementation and API for the DDS communication. This will be further elaborated in the next section, [Developer Navigation Stack](#).

Developer Navigation Stack

In this implementation, it is assumed that the software running on the mobile robot was written by the robot developers themselves (or their immediate subcontractors) and the developers fully understand and have access to their robot's internal control software, API's and interfaces. This level of understanding and access will be necessary for implementing your own `free_fleet` client wrapper. The block diagram below illustrates this configuration:



Once the developer's `free_fleet` client is fully functional, it will be a simple task of launching the same ROS 2 `free_fleet` server as mentioned earlier in this section to work with the fleet

adapters through ROS 2 messages and topics.

Found a bug? [Edit this page on GitHub.](#)

Read-only Fleet Integration

In this section, we will cover the prototype API for integrating the **Read Only** category of mobile robot fleets. This means we assume the mobile robot fleet manager only allows RMF to see updates about where its robots are located and where they intend to go, but it does not offer any control over where the robots are going or how they can move around. This type of adapter is primarily aimed at legacy systems that were developed before RMF and did not anticipate the possibility of a third-party being able to command the robots.

Fleet Driver API

The Fleet Driver API was an experimental API developed in the early stages of the RMF research project. It can still be used for a read-only fleet adapter implementation until an officially supported C++ API comes out to replace it.

The Fleet Driver API uses ROS 2 messages from the `rmf_fleet_msgs` package. To use this API, you will want to write a ROS 2 application (using either `rcldcpp` or `rcldpy`) which we will refer to as the *Fleet Driver*. The job of the Fleet Driver is to transmit `rmf_fleet_msgs/FleetState` messages out to the `fleet_states` topic.

Inside the `FleetState` message is the `name` field. Be sure to fill in the correct name for your fleet state. There is also a collection of `rmf_fleet_msgs/RobotState` messages. For integrating a read-only fleet with RMF, the most crucial fields of the `RobotState` message are:

- `name` - The name of the robot whose state is being specified.
- `location` - The current location of the robot.
- `path` - The sequence of locations that the robot will be traveling through.

Inside the `rmf_fleet_msgs/Location` message, the `t` field (which represents time) is generally ignored by the read-only fleet adapter. We assume that it is too cumbersome for your Fleet Driver to make timing predictions, so we have the read-only fleet adapter make the predictions for you based on the traits of the vehicle.

Configuring the Read Only Fleet Adapter

For the prototype read-only integration, there are two applications that need to be launched:

1. The Fleet Driver mentioned above which you write specifically for your fleet's custom API
2. The `read_only` fleet adapter which must be launched through ROS 2

To launch the fleet adapter, you will need to use `ros2 launch` and include `rmf_fleet_adapter/fleet_adapter.launch.xml` file with the required parameters filled in. An example of this using the XML front-end of `ros2 launch` [can be found in `rmf\_demos`](#), copied below:

```
<?xml version='1.0' ?>

<launch>

  <arg name="fleet_name" default="caddy" description="Name of this fleet of caddy robots"/>

  <group>
    <include file="$(find-pkg-share rmf_fleet_adapter)/fleet_adapter.launch.xml">

      <!-- The name and control type of the fleet -->
      <arg name="fleet_name" value="$(var fleet_name)"/>
      <arg name="control_type" value="read_only"/>

      <!-- The nominal linear and angular velocity of the caddy -->
      <arg name="linear_velocity" value="1.0"/>
      <arg name="angular_velocity" value="0.6"/>

      <!-- The nominal linear and angular acceleration of the caddy -->
      <arg name="linear_acceleration" value="0.7"/>
      <arg name="angular_acceleration" value="1.5"/>

      <!-- The radius of the circular footprint of the caddy -->
      <arg name="footprint_radius" value="1.5"/>
      <!-- Other robots are not allowed within this radius -->
      <arg name="vicinity_radius" value="5.0"/>

      <arg name="delay_threshold" value="1.0"/>

    </include>
  </group>
```

The critical parameters are:

- `fleet_name`: This must match the `name` value that the Fleet Driver gives to its `FleetState` messages.
- `control_type`: This must be `"read_only"`.
- `linear_velocity`, `angular_velocity`, `linear_acceleration`, and `angular_acceleration`: These are estimates of the kinematic properties of the vehicles. For the sake of effective scheduling, it is preferable to overestimate these values than

underestimate them, so it is best to think of these parameters as upper bounds for the values.

- `footprint_radius` : The radius of physical space that the vehicle occupies. This should cover the maximum extent of the physical footprint.
- `vicinity_radius` : The radius around the robot in which other robots are forbidden to physically enter. It is assumed that another robot entering this radius will interfere with the ability of this robot to operate.

When the launch file and Fleet Driver application are both ready, you can launch them side-by-side and the integration of the read-only fleet adapter will be complete.

Found a bug? [Edit this page on GitHub](#).

Doors

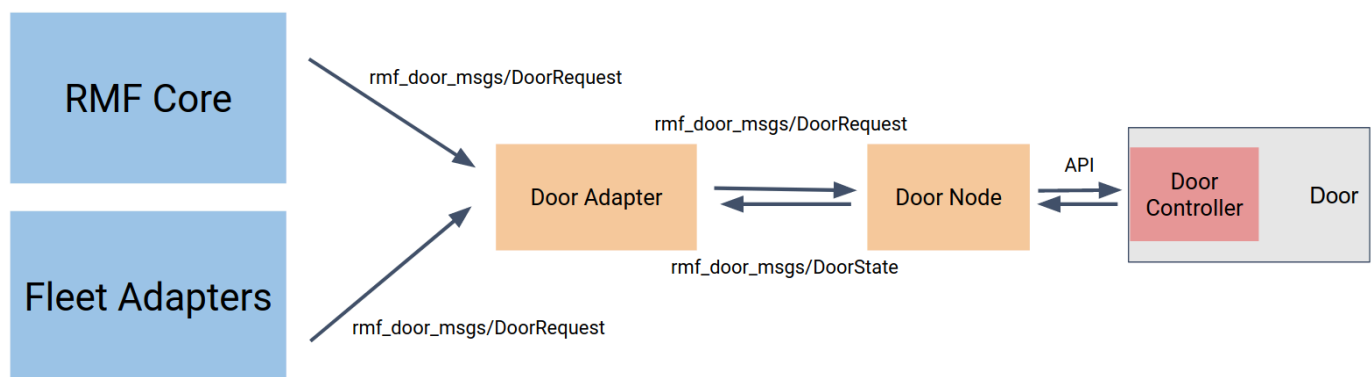
Map Requirements

Before a door can be properly integrated, be sure to draw up the door locations with the correct door names on the navigation graph using `traffic_editor`. The instructions to do so can be found in the [Traffic Editor](#) chapter.

Integration

Door integration is required when integrating RMF into a new environment. For hopefully obvious reasons, only automated doors can be integrated with RMF though it may be possible to trigger an alert to a designated human to open a manual door but this is not recommended. An automated door can be defined as an electronically powered door that is remotely controllable, either using a remote trigger or has been outfitted with a computing unit capable of commanding the door to open and close when needed, using certain interfaces.

Doors can be integrated with RMF using a ROS 2 door node and a door adapter, which we sometimes refer to as a door supervisor. The block diagram below displays the relationship and communication modes between each component:



The door node will have to be implemented based on the make and model of the door that is being integrated in order to address the specific API of the door controller module. The communication protocol will also be dependent on the door and controller model, which might be some form of `REST`, `RPCXML`, etc. The door node is in charge of publishing its state and receiving commands over ROS 2, using the messages and topics listed below:

Message Types	ROS2 Topic	Description
<code>rmf_door_msgs/DoorState</code>	<code>/door_states</code>	State of the door published by the door node
<code>rmf_door_msgs/DoorRequest</code>	<code>/door_requests</code>	Direct requests subscribed by the door node and published by the door adapter
<code>rmf_door_msgs/DoorRequest</code>	<code>/adapter_door_requests</code>	Requests to be sent to the door adapter/supervisor to request safe operation of doors

The door adapter stands in between the rest of the RMF core systems, fleet adapters, and the door node, and acts like a state supervisor ensuring that the doors are not acting on requests that might obstruct an ongoing mobile robot task or accidentally closing on it. It keeps track of the door state from the door node, and receives requests from the `adapter_door_requests` topic which are published by either fleet adapters or other parts of the RMF core system. Only when the door adapter deems that a request is safe enough to be performed, it will instruct the door node using a request. It should also be noted that direct requests sent to the door node, without going through the door adapter will be negated by the door adapter, to return it to its prior state in order to prevent disruptions during operations with mobile robots.

Door Adapter Template

To make the process of integrating doors with RMF even simpler, we have open-sourced a [template package](#) where users only need to update certain blocks of code with the API calls to their specific door controller.

Found a bug? [Edit this page on GitHub](#).

Lifts (i.e. Elevators)

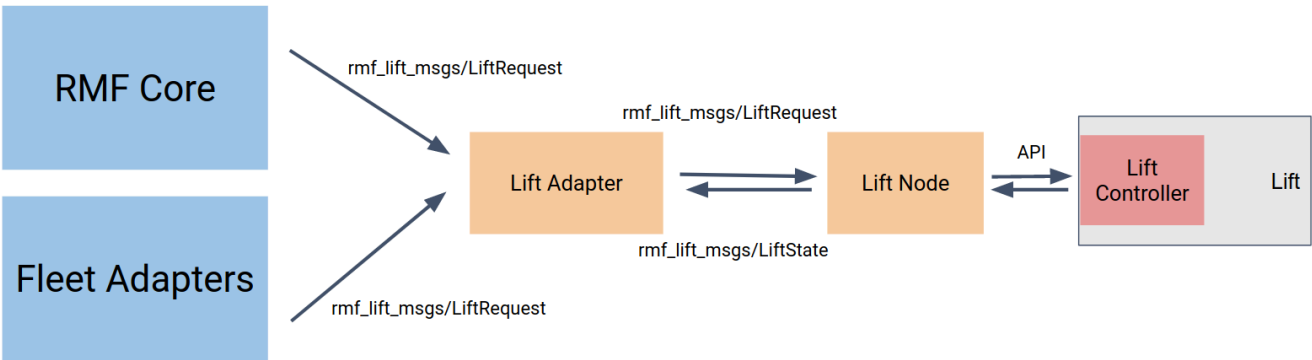
Map requirements

Before a lift can be properly integrated, be sure to draw up the lift locations with the correct lift names and levels on the navigation graph using `traffic_editor` . The instructions to do so can be found in [Traffic Editor](#) chapter.

Integration

Elevator integration will allow RMF to work over multiple levels, resolving conflicts and managing shared resources on a larger scale. Similar to door integration, the basic requirement is that the lift controller accepts commands using a prescribed protocol, `opc` is one such example.

The elevators will be integrated in a similar fashion as doors as well, relying on a lift node and a lift adapter. The following block diagram shows how each component works with each other:



The lift node will act as a driver to work with the lift controller. An example of a lift node can be found in this [repository](#). The node will publish its state and receive lift requests over ROS 2, using the messages and topics listed below.

Message Types	ROS2 Topic	Description
<code>rmf_lift_msgs/LiftState</code>	<code>/lift_states</code>	State of the lift published by the lift node

Message Types	ROS2 Topic	Description
<code>rmf_lift_msgs/LiftRequest</code>	<code>/lift_requests</code>	Direct requests subscribed by the lift node and published by the lift adapter
<code>rmf_lift_msgs/LiftRequest</code>	<code>/adapter_lift_requests</code>	Requests to be sent to the lift adapter/supervisor to request safe operation of lifts

A lift adapter subscribes to `lift_states` while keeping track of the internal and desired state of the lift in order to prevent it from performing any actions that might interrupt mobile robot or normal operations. The lift adapter performs this task by receiving lift requests from the fleet adapters and the RMF core systems and only relaying the instructions to the lift node if it is deemed appropriate. Any requests sent directly to the lift node, without going through the lift adapter, will also be negated by the lift adapter, to prevent unwanted disruption to mobile robot fleet operations.

Found a bug? [Edit this page on GitHub](#).

Workcells

Currently RMF has 2 types of sample workcells, namely: `Dispenser` and `Ingestor`.

Message Types	ROS2 Topic	Description
<code>rmf_dispenser_msgs/DispenserRequest</code>	<code>/dispenser_requests</code>	Direct requests subscribed by the dispenser node
<code>rmf_dispenser_msgs/DispenserResult</code>	<code>/dispenser_results</code>	Result of a dispenser request, published by the dispenser
<code>rmf_dispenser_msgs/DispenserState</code>	<code>/dispenser_states</code>	State of the dispenser published by the dispenser periodically
<code>rmf_ingestor_msgs/IngestorRequest</code>	<code>/ingestor_requests</code>	Direct requests subscribed by the ingestor node
<code>rmf_ingestor_msgs/IngestorResult</code>	<code>/ingestor_results</code>	Result of a ingestor request, published by the ingestor
<code>rmf_ingestor_msgs/IngestorState</code>	<code>/ingestor_states</code>	State of the dispenser published by the ingestor periodically

In `rmf_demos` world, both `TeleportDispenser` and `TeleportIngestor` [plugins](#) act as workcell adapter nodes.

Workcells currently work alongside with Delivery Task. In `fleet_adapter.launch.xml`, `perform_deliveries` needs to be `true` for the robot to accept a delivery task.